

React Learning Curriculum

Personal Track • Starting from HTML, CSS, JavaScript/TypeScript, DOM & BOM

Goal: progress from being comfortable building webpages with JavaScript to building, testing, and structuring maintainable React + TypeScript applications.

Starting Point

- HTML and CSS fundamentals
- JavaScript / TypeScript fundamentals
- DOM and BOM familiarity
- Able to build simple websites, with occasional hiccups

Recommended Pace

Estimated duration: 8–12 weeks at roughly 1–2 hours/day, 5 days/week.

Core philosophy: Learn → build a tiny example → deliberately break it → fix it → apply it in a project.

Phase 0 — React Mental Model

Duration: 1–2 days

Learn

- Declarative vs imperative UI
- Components, props, state, rendering and re-rendering
- Component trees, JSX, one-way data flow
- Thinking in terms of what the UI should look like for a given state

Build

- Counter with increment, decrement, reset, and configurable starting value

Phase 1 — JSX & Components

Duration: ~1 week

Learn

- JSX syntax and expressions
- Conditional rendering and lists
- Keys and fragments
- Components, composition, props, children, event handlers

Build

- Profile card

- Product listing
- Simple landing page

Phase 2 — State & Interaction

Duration: ~1 week

Learn

- useState
- State updates and functional updates
- State as a snapshot
- Controlled inputs
- Lifting state up
- State ownership
- Derived state

Build

- Todo application with add/delete/complete/edit/filter
- Shopping cart with quantities and derived totals

Phase 3 — Forms & Realistic UI

Duration: ~1 week

Learn

- Controlled vs uncontrolled inputs
- Form submission and validation
- Multiple input types
- Error, loading, disabled, success, and empty states

Build

- User registration form with validation and simulated submission

Phase 4 — Effects & External Systems

Duration: ~1 week

Learn

- useEffect
- Dependency arrays and cleanup
- Synchronizing with external systems
- Timers and event listeners
- Fetching data
- When not to use useEffect

Build

- API search application
- Window-information app using browser APIs

Phase 5 — Data Fetching & Application Architecture

Duration: 1–2 weeks

Learn

- HTTP and JSON lifecycle
- Async/await and fetch
- Loading/error/success states
- Pagination, search, filtering
- API abstraction
- Reasonable project structure

Build

- Movie Explorer with search, filters, sorting, pagination, and detail pages

Phase 6 — React Router

Duration: 3–4 days

Learn

- Routes and nested routes
- Route and query parameters
- Navigation and links
- `useNavigate`, `useParams`, `useSearchParams`
- Layouts and 404 pages

Build

- Upgrade Movie Explorer into a multi-route SPA

Phase 7 — TypeScript + React

Duration: ~1 week

Learn

- Typed props and events
- Typed `useState`/`useRef`
- API response types
- Generic components and hooks
- Union and discriminated-union patterns
- `ReactNode` and utility types

Build

- Properly convert an existing project to TypeScript rather than merely renaming files

Phase 8 — Reusable Logic & Custom Hooks

Duration: ~1 week

Learn

- Custom hooks and hook rules
- Reusable stateful logic
- useRef
- useMemo/useCallback and when they are actually useful

Build

- useFetch
- useDebounce
- useLocalStorage
- useWindowSize

Phase 9 — Global State

Duration: ~1 week

Learn

- Lifting state
- Context and its limitations
- Global store concepts
- Zustand: stores, actions, selectors, subscriptions

Build

- E-commerce frontend with cart, session/user state, and local UI state

Phase 10 — Professional React

Duration: 1–2 weeks

Learn

- Rendering behavior and React DevTools
- Performance and code splitting
- Error boundaries and fallback UI
- Accessibility
- Vitest and React Testing Library
- Behavior-focused testing

Build

- Test and improve one existing application

Phase 11 — Server State

Duration: ~1 week

Learn

- TanStack Query
- Queries and mutations
- Caching and stale data
- Invalidation/refetching
- Pagination
- Optimistic updates

Build

- API-driven dashboard using server-state patterns

Phase 12 — Full-Stack React Project

Duration: 2–3 weeks

Learn

- React + TypeScript frontend
- REST API integration
- Authentication
- CRUD, pagination, filtering, search
- Authorization
- Frontend architecture

Build

- Portfolio-quality Project Management System backed by Node.js and PostgreSQL

Advanced — React Internals

Duration: Optional

Learn

- Reconciliation and Fiber
- Rendering vs committing
- Batching
- State preservation and keys
- Scheduling
- Suspense
- Server Components concept

Build

- Use this phase to explain confusing React behavior rather than as a prerequisite for normal app development

Project Ladder

Stage	Project	Primary Skill
0	Counter	React mental model
1	Profile / Landing Page	Components + JSX
2	Todo	State
2	Shopping Cart	State architecture
3	Registration Form	Forms
4	API Search	Effects + async
5	Movie Explorer	API architecture
6	Movie Explorer v2	Routing
7	TypeScript migration	TypeScript
8	Custom hooks	Reusable logic
9	E-commerce	Global state
10	Tested app	Professional practices
11	API dashboard	Server state
12	Project Management System	Full-stack engineering

What Not to Learn Too Early

Avoid starting with Redux, Next.js, Tailwind, component libraries, TanStack Query, Zustand, complex state-management patterns, elaborate folder architectures, Server Components, SSR/SSG, or micro-frontends. Learn these after the core React mental model is solid.

Core Reasoning Questions

- Why is this value state rather than a normal variable?
- What causes this component to render again?
- What data belongs to this component, and what should be owned by its parent?
- Is this value actually derived from existing state?
- Is this effect synchronizing with an external system, or am I using it unnecessarily?
- Is this client state or server state?
- Does this component boundary make the application easier to reason about?
- Am I abstracting reusable behavior, or merely creating abstractions because I can?

Target Outcome

By the end, you should be able to build a React + TypeScript application from scratch, reason about state and data flow, consume and structure API data, implement routing and forms, choose appropriate state-management approaches, test important behavior, and integrate the frontend with a Node.js/PostgreSQL backend.

The objective is not to memorize React APIs. It is to develop a reliable way of reasoning about UI, state, data flow, and component boundaries.